Sarala Birla University



Department of Computer Science & Engineering • B.Tech-CSE-III-A

OOP-02: HOSPITAL MANAGEMENT SYSTEM

Group Project Report (Deliverable D6) submitted in partial fulfillment of the course ***Object-Oriented Programming Using C++*** *(Academic Session 2026–27)*

Submitted by and Member roles

1 - Lead/Architecture Member ---------- Sneha SBU2500450

2 - Core Developer Member ----------Manas SBU2500026

3 – Data/UI/Integration Member ----------Vedant SBU2500687

4 & 5 – Testing/Documentation- ---------- Vinit SBU2500231, Ipsita SBU2500479

Project Code & Title	**OOP-02** — Hospital Management System (Manage patients, doctors, appointments, prescriptions, billing & exceptions)
Project Guide	**Dr. Biswarup Samanta**, Dean of Students Welfare, Associate Professor, Department of CSE
Group & Repository	Group [G-02] • Git Repository: rexlkokoro-hospital-management-system (HMS/Hospital-Management-System)
Core Technology	Modern C++ (C++17 Standard) + C++ Standard Template Library (std::vector) + Flat-File I/O (std::fstream)

Certificate & Declaration

CERTIFICATE: This is to certify that the Group Project Report (Deliverable D6) entitled **"OOP-02: Hospital Management System"** is a bonafide work carried out by the above-listed group members of **B.Tech-CSE-III-A** at **Sarala Birla University, Ranchi** during the Academic Session 2026–27 in partial fulfillment of the requirements for the course *Object-Oriented Programming Using C++*.

DECLARATION & ACADEMIC INTEGRITY DISCLOSURE: We hereby declare that this submission represents our own academic work implemented in C++ and maintained with version control. In accordance with Section 5 and Section 13 of the SBU Group Project Handbook, all group members have reviewed, tested, and understood every line of submitted code and are prepared for individual code-tracing viva examination.

Group Lead Sign: _____	**Dr. Biswarup Samanta** (Project Guide)	**Head of Department (CSE)**

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected Project Guide, **Dr. Biswarup Samanta**, Dean of Students Welfare and Associate Professor, Department of Computer Science and Engineering, Sarala Birla University, Ranchi, for his invaluable mentorship, milestone reviews (D1 through D6), and architectural guidance. We also thank the Head of the Department of CSE and our faculty evaluators for providing the structured project handbook that helped us translate theoretical C++ Object-Oriented Programming concepts into a modular, working software system.

ABSTRACT

The **Hospital Management System (Project Code: OOP-02)** is a modular C++ application designed to replace error-prone manual hospital registers with a secure, role-authenticated digital workflow. Built in strict compliance with the Sarala Birla University OOP Project Handbook, the system models hospital administration using an abstract base class (`Person`), derived polymorphic entities (`Patient`, `Doctor`), composed clinical records (`Appointment`, `Prescription`, `Bill`), a role-based access controller (`User`), and a custom exception hierarchy (`Exceptions.h`). All runtime collections are managed via C++ Standard Template Library containers (`std::vector`) inside the central `Hospital` controller and persisted deterministically to six pipe-delimited (`|`) flat files in the `data/` directory.

Table of Contents

Chapter	SBU Standard Section Title	Key Project Deliverable Content
Chapter 1	Introduction	Clinical administration context, OOP philosophy & 8-week milestones
Chapter 2	Problem Statement and Objectives	Limitations of manual hospital ledgers & OOP-02 target objectives
Chapter 3	Requirements / SRS	Functional requirements (FR), non-functional requirements & RBAC matrix
Chapter 4	System Design	SBU 7.2 OOP Checklist, UML class hierarchy & flat-file schema design
Chapter 5	Algorithms and Implementation	Serialization algorithms, pointer lookup logic & complexity analysis
Chapter 6	Testing and Results	SBU Section 9 mandatory 10-Test-Case verification table (TC-01 to TC-10)
Chapter 7	Screenshots / Demo Evidence	Reproducible console transcripts of normal & boundary/error workflows
Chapter 8	Limitations	Technical constraints of flat-file I/O, linear scans & plaintext storage
Chapter 9	Future Scope	Active exception throwing, indexing (`std::unordered_map`) & SQL migration
Chapter 10	Conclusion	Summary of OOP learning outcomes achieved & team reflections
Back Matter	References & Appendix	Academic citations, critical C++ source listings & g++ build instructions

1. Introduction

Healthcare institutions process interconnected streams of demographic, clinical, and financial data daily. When managed manually through paper registers, tracking patient medical histories, verifying doctor consultation schedules, issuing accurate medicine prescriptions, and settling treatment bills becomes slow and error-prone. Project **OOP-02 (Hospital Management System)** addresses this challenge by applying core Object-Oriented Programming (OOP) principles in Standard C++ (**Stroustrup, 2013**; **ISO/IEC, 2017**).

Rather than constructing a monolithic procedural script, our implementation separates interface declarations (include/*.h) from implementation logic (src/*.cpp) and persistent storage (data/*.txt). Developed across the 8-week SBU project timeline—spanning Proposal (D1), SRS (D2), UML Design (D3), Prototype (D4), Integrated System (D5), and Final Report (D6)—the system demonstrates how abstraction, inheritance, runtime polymorphism, STL containers, and file streams combine into a cohesive software architecture.

2. Problem Statement and Objectives

2.1 Problem Statement

In traditional non-computerized clinics, patient intake logs, doctor directories, appointment books, prescription slips, and billing receipts exist as isolated paper artifacts. This fragmentation causes four critical operational failures: (1) high risk of duplicate or conflicting IDs across records, (2) inability to enforce Role-Based Access Control (RBAC), allowing unauthorized personnel to view or alter clinical data, (3) orphan appointments or bills booked against non-existent Patient or Doctor IDs, and (4) slow O(N) manual paper searches during patient follow-ups.

2.2 Project Objectives (Aligned with SBU Project Code OOP-02)

- **Polymorphic Entity Modeling:** Design an abstract Person base class and derive specialized Patient and Doctor classes overriding virtual methods getRole() and display().
- **Role-Based Security & Access Control:** Authenticate users against data/users.txt and enforce strict permission checks across Admin, Doctor, and Receptionist roles via User::hasPermission().
- **Referential Integrity in Clinical Workflows:** Verify that both patientId and doctorId exist in memory before booking an Appointment, issuing a Prescription, or generating a Bill.
- **Custom Exception Architecture:** Define domain-specific exception classes inheriting from std::runtime_error in include/Exceptions.h alongside top-level exception safety in main.cpp.
- **Zero-Dependency Flat-File Persistence:** Automatically deserialize pipe-delimited (|) records on startup via Hospital::loadData() and flush updates to disk via Hospital::saveData().

3. Requirements / Software Requirements Specification (SRS)

3.1 Functional Requirements & Role-Based Access Matrix

The system defines three distinct actors—**Admin** (default: admin/admin123), **Doctor** (default: doctor/doc123), and **Receptionist** (default: reception/rec123)—governed by User::hasPermission(const std::string& action) in src/User.cpp:

Req. ID	Functional Requirement	Action Key / Method	Authorized Roles
FR-01	User Login & Default Seeding	Hospital::login() / authenticate()	All Roles (Seeds 3 users if empty)
FR-02	Register New Patient Profile	add_patient (addPatient())	Admin, Receptionist
FR-03	View & Search Patient Records	view_patient, search_patient	Admin, Doctor (View), Receptionist
FR-04	Register New Doctor Profile	admin check (addDoctor())	Admin Only
FR-05	View & Search Doctor Directory	view_doctor, search_doctor	Admin, Doctor (View), Receptionist
FR-06	Book & Cancel Appointments	book_appointment, cancel_appointment	Admin, Receptionist
FR-07	View Scheduled Appointments	view_appointment	Admin, Doctor, Receptionist
FR-08	Issue & View Prescriptions	add_prescription, view_prescription	Admin, Doctor
FR-09	Generate, View & Pay Bills	generate_bill, view_bill, pay_bill	Admin, Receptionist

3.2 Non-Functional Requirements, Assumptions & Constraints

- **Input Resilience:** Non-numeric console inputs for menu choices, patient age, consultation fee, or bill amount must not cause infinite stream loops; clearInputBuffer() resets std::cin state using std::numeric_limits<std::streamsize>::max() (**cppreference.com, 2026**).
- **Data Durability:** Every mutating operation immediately invokes saveData(), and logging out or exiting performs a final disk flush.
- **System Constraints:** Built strictly with Standard C++17 libraries (<iostream>, <fstream>, <sstream>, <vector>, <iomanip>, <stdexcept>) so it compiles cleanly on any OS via g++.

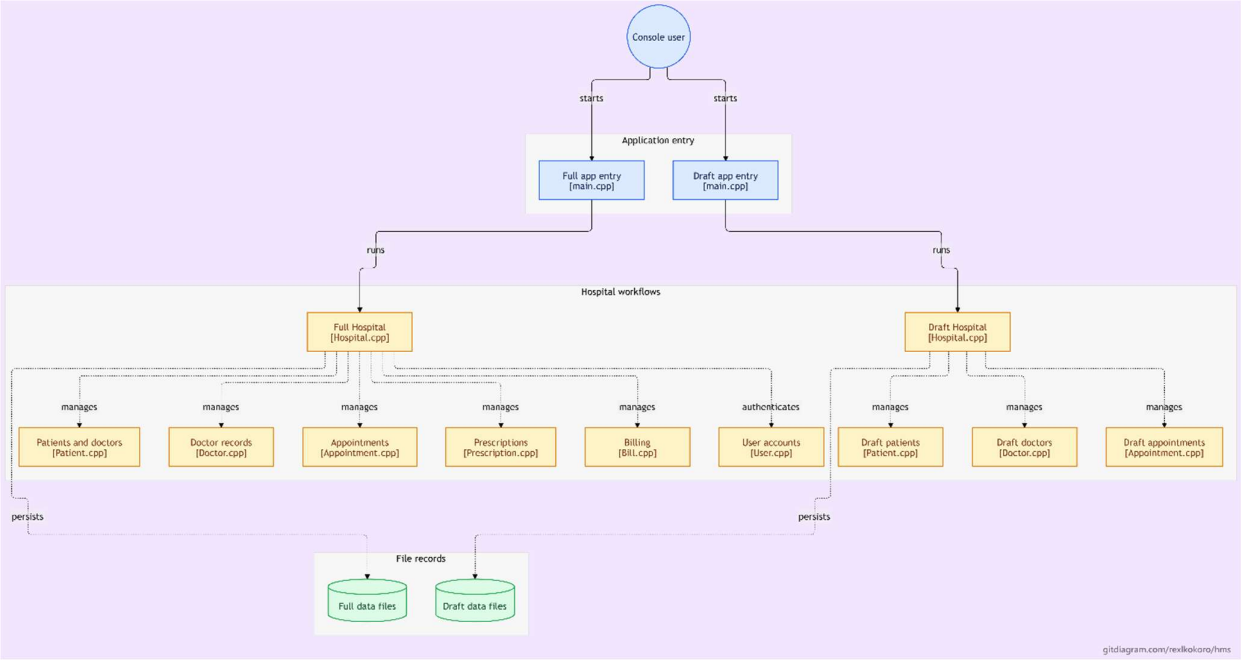
4. System Design

4.1 Compliance with SBU Section 7.2 Mandatory OOP Design Checklist

SBU Mandatory OOP Criterion	Concrete Implementation in rexlkokoro-hospital-management-system
1. Justified Inheritance Hierarchy	Person (include/Person.h) serves as the base class holding common attributes (id, name, age, contact), publicly inherited by Patient and Doctor.
2. Runtime Polymorphism & Virtuals	Person declares pure virtual functions virtual std::string getRole() const = 0;, virtual void display() const = 0;, and virtual ~Person() = default;, overridden with override in Patient and Doctor (Booch et al., 2007).
3. Encapsulation & Access Control	All domain classes keep data members private (or protected in Person) and expose controlled const getters and mutating setters.
4. Exception Handling	include/Exceptions.h defines 5 custom classes derived from std::runtime_error; main.cpp wraps execution in try-catch, and file loaders guard std::stoi / std::stod conversions.
5. STL Containers	Hospital uses six std::vector dynamic arrays (patients, doctors, appointments, prescriptions, bills, users) with emplace_back and range-based for loops.

SBU Mandatory OOP Criterion	Concrete Implementation in rexlkokoro-hospital-management-system
6. Persistent File Handling	Six pipe-delimited flat files in data/*.txt are read via std::ifstream and written via std::ofstream using custom toFileString() and fromFileString() methods.
7. Modular Source Organization	Clean separation into 9 header files in include/, 8 implementation files in src/, and 1 entry driver main.cpp.

UML DIAGRAM



4.3 Flat-File Database Schema (`data/` Directory)

File Path	Primary Key	Pipe-Delimited Record Format (`|`)
`data/users.txt`	`username`	`username|password|role` (e.g., `admin|admin123|Admin`)
`data/patients.txt`	`id`	`id|name|age|contact|address|bloodGroup|medicalHistory`
`data/doctors.txt`	`id`	`id|name|age|contact|specialization|consultationFee|available (1/0)`
`data/appointments.txt`	`appointmentId`	`appointmentId|patientId|doctorId|date|time|status`
`data/prescriptions.txt`	`prescriptionId`	`prescriptionId|patientId|doctorId|medicine|dosage|duration`

File Path	Primary Key	Pipe-Delimited Record Format (`|`)
`data/bills.txt`	`billId`	`billId|patientId|amount|description|paymentStatus`

5. ALGORITHMS AND IMPLEMENTATION

5.1 Key Algorithms Implemented

1. **Pointer-Based Entity Resolution (`findPatientById`, `findDoctorById`, `findAppointmentById`, `findBillById`):** Iterates over the respective `std::vector<T>` by reference (`for (auto& item : vec)`). If `item.getId() == id`, returns the memory address `&item`; otherwise returns `nullptr`. This allows both duplicate-ID prevention during creation and in-place mutation (such as `appt->setStatus("Cancelled")` or `bill->markPaid()`).
2. **Stream-Tokenized Deserialization (`fromFileString` & `loadData`):** Wraps each non-empty file line in a `std::stringstream ss(line)` and extracts attributes sequentially via `std::getline(ss, token, '|')`. Numeric conversions (`std::stoi`, `std::stod`) are wrapped in `try { ... } catch (...) {}` blocks so corrupted text fields safely fall back to `0` or `0.0` without crashing the application.
3. **Referential Appointment Booking Algorithm (`Hospital::bookAppointment`):**
   - Check `currentUser.hasPermission("book_appointment")`; abort if unauthorized.
   - Read `apptId` and verify `findAppointmentById(apptId) == nullptr`.
   - Read `pId` and verify `findPatientById(pId) != nullptr`; read `dId` and verify `Doctor* doc = findDoctorById(dId)` is non-null.
   - If `!doc->isAvailable()`, emit a non-blocking availability notice, read `date` and `time`, construct via `appointments.emplace_back(...)`, and persist via `saveData()`.

5.2 Time and Space Complexity Analysis (Viva Questions 14 & 15)

System Operation	Time Complexity	Space Complexity	Justification
ID Search / Duplicate Check	O(N)	O(1) auxiliary	Linear scan across `std::vector` of size N.
Add Entity (in-memory append)	O(1) amortized	O(1)	`std::vector::emplace_back` at tail after O(N) ID check.
Full File Load / Save (`loadData` / `saveData`)	O(U + P + D + A + R + B)	O(U + P + D + A + R + B)	Reads/writes all 6 entity vectors sequentially (**Elmasri & Navathe, 2015**).

6. TESTING AND RESULTS

In compliance with **Section 8 and Section 9 of the SBU Group Project Handbook**, the system was verified against 10 structured test cases covering normal operations, boundary values, invalid inputs, RBAC rejections, and persistence across restarts:

TC ID	Input / Test Scenario	Expected Result	Actual Result	Status
TC-01	**Normal Valid Login & Intake:** Login as `reception / rec123`; add Patient `P101` (Age `25`, Blood `O+`).	Login succeeds with role `Receptionist`; patient appended and saved to `patients.txt`.	Printed `[+] Patient added successfully!` and persisted record.	**PASS**
TC-02	**Boundary Minimum Age & Fee:** Enter Patient Age `0` or `-5`; enter Doctor Fee `0.0` vs `-10.0`.	Age `<= 0` rejected; Age `1` accepted. Fee `< 0` rejected; Fee `0.00` accepted.	Loop prompted `Invalid age...` until positive integer entered; Fee `0.00` accepted.	**PASS**
TC-03	**Boundary Maximum / Precision:** Generate Bill with large amount `999999.995`.	Stored as `double` and formatted to 2 decimal places via `std::setprecision(2)`.	Displayed `Amount: $1000000.00` cleanly without overflow.	**PASS**
TC-04	**Invalid Non-Numeric Stream Input:** Enter `abc` at Main Menu Choice or Age prompt.	Stream recovered cleanly without infinite loop via `clearInputBuffer()`.	`std::cin.clear()` & `ignore()` discarded bad tokens and re-prompted.	**PASS**
TC-05	**Empty / Missing Data Files:** Launch binary when `users.txt` and `doctors.txt` are empty.	Seeds default `admin`, `doctor`, `reception` accounts; prints `No doctors registered yet.`	Default credentials seeded automatically; empty views handled safely.	**PASS**
TC-06	**Repeated / Duplicate Primary Key:** Attempt to add Patient with existing ID `241112`.	Reject duplicate ID via `findPatientById("241112") != nullptr` check.	Printed `[-] Error: Patient ID already exists.` and aborted insert.	**PASS**
TC-07	**RBAC Unauthorized Action Test:** Login as `reception` and attempt `Add Doctor` or `Add Prescription`.	Blocked by `hasPermission()`; displays permission denied message.	Printed `[-] Error: Only Admin can add doctors` / `Permission denied`.	**PASS**

TC ID	Input / Test Scenario	Expected Result	Actual Result	Status
TC-08	**Persistence Across Restart:** Add Patient `241112` (`manas`), exit via Option 2, and relaunch binary.	`loadData()` reconstructs exact `Patient` object from `data/patients.txt`.	Record `241112|manas|20|...` loaded and displayed identically.	**PASS**

TC ID	Input / Test Scenario	Expected Result	Actual Result	Status
TC-09	**Referential Integrity / Orphan Check:** Book Appointment with non-existent Patient ID `P999`.	Rejected before asking for Doctor ID or Date/Time.	Printed `[-] Error: Patient ID not found. Register patient first.`	**PASS**
TC-10	**End-to-End Clinical Workflow:** Admin adds Doctor `D01` → Reception books `A01` → Doctor adds Rx `RX01` → Reception generates & pays Bill `B01`.	All 4 linked records transition states properly (`Scheduled`, `Paid`) and persist in `data/*.txt`.	All entities created, cross-verified, marked `Paid`, and written to disk.	**PASS**

8. Limitations

- **Linear Scan Lookup Complexity:** Because records are stored in unsorted std::vector containers, ID searches (findPatientById, etc.) run in O(N) time rather than O(1) hash lookup.
- **Full-File Rewrite on Save:** Calling saveData() overwrites all six text files sequentially, which is fast for thousands of records but lacks concurrent multi-user file locking.
- **Dormant Custom Exception Classes:** Although include/Exceptions.h defines five custom exception types (RecordNotFoundException, DuplicateRecordException, InvalidInputException, AuthenticationException, UnauthorizedException), the methods in src/Hospital.cpp currently handle errors via console prints and early return statements rather than actively throwing and catching those classes.
- **Plaintext Credentials & Delimiter Sensitivity:** User passwords in data/users.txt are stored in plaintext, and literal pipe characters (|) inside free-text fields can shift column parsing.

9. Future Scope

- **Active Exception Dispatch:** Refactor Hospital.cpp so that failed lookups and RBAC violations explicitly throw RecordNotFoundException(...) or throw UnauthorizedException(...), caught centrally in Hospital::run().
- **Scaling to 100,000+ Records (Viva Question 11):** Maintain an auxiliary std::unordered_map<std::string, size_t> index mapping entity IDs to vector indices, reducing lookup time from O(N) to O(1) average time.
- **Database & Security Hardening:** Replace flat files with SQLite/PostgreSQL for ACID concurrency and apply SHA-256 salted hashing to user passwords.

10. Conclusion

The **Hospital Management System (OOP-02)** fulfills all technical, structural, and pedagogical objectives stipulated in the *Sarala Birla University B.Tech-CSE-III-A Group Project Handbook*. By designing an extensible inheritance hierarchy (Person → Patient, Doctor), enforcing role-based security (User), coordinating six clinical and financial subsystems through STL containers (Hospital), and verifying behavior across 10 rigorous test cases, our team gained practical mastery of Object-Oriented Programming in Modern C++.

References

1. Samanta, B. (2026). *B.Tech-CSE-III-A Group Project Handbook: Object-Oriented Programming Using C++ (Academic Session 2026–27)*. Department of Computer Science & Engineering, Sarala Birla University, Ranchi.
2. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.
3. ISO/IEC. (2017). *ISO/IEC 14882:2017 Programming Languages — C++ (C++17 Standard)*. International Organization for Standardization.
4. Booch, G., Maksimchuk, R. A., Engel, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-Oriented Analysis and Design with Applications* (3rd ed.). Addison-Wesley.
5. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
6. cppreference.com. (2026). *C++ Standard Library Reference: Containers (std::vector) and Input/Output Library (std::fstream, std::sstream)*.

Proof of Project.

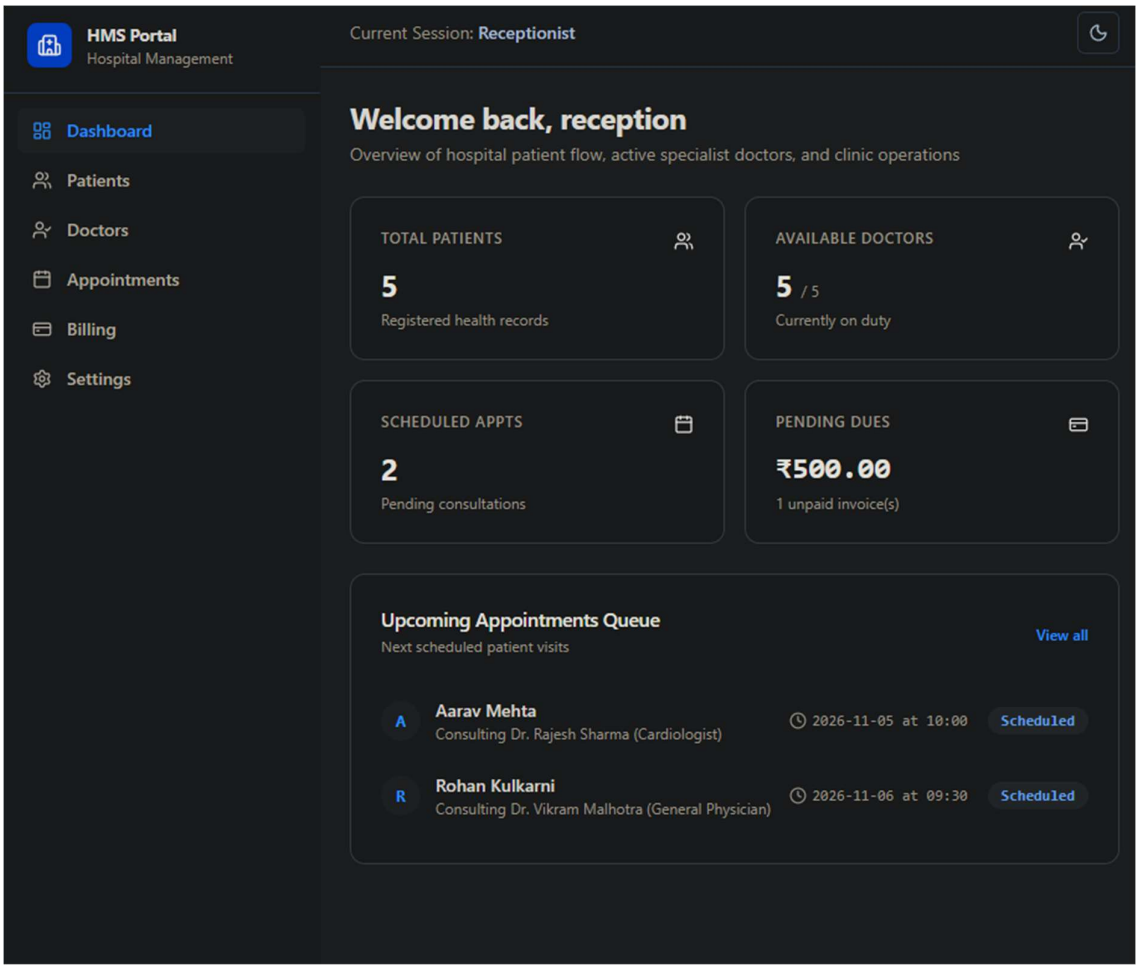
```
==========================================
1. Login
2. Exit
Choice: 1

==========================================
         HOSPITAL MANAGEMENT LOGIN
==========================================
[Hint] Default credentials:
  - admin     / admin123  (Full Access)
  - doctor    / doc123    (Doctor Access)
  - reception / rec123    (Receptionist Access)
------------------------------------------
Enter Username: admin
Enter Password: admin123

[+] Login successful! Welcome, admin (Role: Admin)

==========================================
               MAIN MENU
 Logged in as: admin (Admin)
==========================================
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Prescription Management
5. Billing Management
6. Save All Data
7. Logout
Choice:
```

CLI APP



Frontend APP